



Fetch & Render Tool

This tool provides a web interface to fetch a URL with various configurations, render the page using a real browser engine, and display the results, including HTTP headers, rendered HTML, and a full-page screenshot. It is designed to accurately simulate how a specific client, like a Googlebot, would see and render a webpage.

☀ Features

- **Customizable Fetching:** Configure the User-Agent, Accept-Language header, and post-load timeout.
- **Accurate JavaScript Rendering:** Utilizes a full browser engine via Selenium to render modern, JavaScript-heavy pages.
- **Desktop & Mobile Simulation:** Fetch pages as either a desktop or smartphone Googlebot, correctly applying device-specific properties.
- **Full-Page Screenshots:** Intelligently scrolls the entire page and stitches together a complete, seamless screenshot, capturing all lazy-loaded content.
- **Robots.txt Awareness:** Option to check for and report resources that would be blocked by a site's `robots.txt` rules.
- **Detailed Results:** View HTTP response headers, the final rendered HTML source code, and a high-fidelity visual representation of the page.
- **Interactive Web UI:** A user-friendly interface built with Flask for easy input and clear presentation of results.



How It Works

The application is composed of several key Python scripts and HTML templates that work together:

1. **Flask Web Server** (This is the core of the application. It serves the main HTML form (`index.html`) and handles the form submission. When a URL is submitted, it gathers all the user-defined settings (User-Agent, language, etc.) and passes them to the fetching module. After the fetch is complete, it renders the results using the `results.html` template.
2. **Fetcher Engine** (This is the powerhouse of the tool. It uses `selenium-wire` to launch a fresh Chrome browser instance for each request with the specified configurations. It navigates to the target URL, waits for the page to load, and then executes a

sophisticated scrolling and screenshotting mechanism. It takes multiple viewport-sized screenshots while scrolling down the page until it reaches the bottom, then stitches them together to create a single, seamless image. It also captures the final rendered HTML, HTTP headers, and other diagnostic data.

3. **HTML Templates** (These files define the structure and appearance of the user interface. `index.html` provides the input form, and `results.html` presents the fetched data in a clean, tabbed layout.
4. **Configuration** (This file stores important path information, such as the project's root directory.

Setup and Installation Guide

This guide provides detailed, step-by-step instructions to get the Fetch & Render tool running on your local machine.

Prerequisites

Before you begin, ensure you have the following installed on your computer:

- **Python (Version 3.7 or newer):** The programming language the tool is built on.
 - To check if you have Python, open your terminal or Command Prompt and type: `python --version` OR `python3 --version`. If you don't have it, download it from the [official Python website](https://www.python.org/).
-
- **Google Chrome Browser:** The tool uses Chrome to render pages.
 - You must have the standard Google Chrome browser installed. You can download it from [google.com/chrome](https://www.google.com/chrome).
-

Step 1: Organize Your Project Files

Create a new folder on your computer to hold all the project files. For example, you can create a folder named `FetchAndRenderTool`. Place all the files you have into this folder:

- `app.py`
- `fetcher.py`
- `config.py`
- `requirements.txt`

- A folder named `templates` which contains:
 - `index.html`
 - `results.html`
-

Step 2: Set Up a Python Virtual Environment

Using a virtual environment is highly recommended as it keeps the project's dependencies isolated from other Python projects on your system.

1. Open your terminal or Command Prompt.
2. Navigate to your project folder using the `cd` command.
3. **codeBash**

Example:

4. **cd** `C:\Users\YourName\Documents\FetchAndRenderTool`
- 5.
6. Create the virtual environment.
7. **codeBash**

On Windows

```
python -m venv venv
```

On macOS or Linux

8. `python3 -m venv venv`
9. This will create a new `venv` folder inside your project directory.
10. Activate the virtual environment. You must do this every time you work on the project.
11. **codeBash**

On Windows

```
venv\Scripts\activate
```

On macOS or Linux

12. `source venv/bin/activate`

13. When activated, you will see `(venv)` at the beginning of your terminal prompt.

Step 3: Install Required Libraries

All the necessary Python libraries are listed in the `requirements.txt` file.

1. Make sure your virtual environment is active (you see `(venv)` in your prompt).
2. Run the following command to install all dependencies at once:
3. `codeBash`
4. `pip install -r requirements.txt`
5. This will download and install Selenium, Flask, Pillow, and all other required libraries into your virtual environment.

Running the Tool

After completing the setup, you are ready to launch the web application.

1. Make sure you are in the project's root directory and your virtual environment is activated (`(venv)` is visible in your prompt).
2. Run the following command to start the Flask web server:
3. `codeBash`
4. `flask --app app run --debug`
 - The `--debug` flag enables debug mode, which is helpful for development as it automatically reloads the server when you make code changes.
- 5.
6. Your terminal will show output indicating that the server is running. It will provide a local URL, which is typically:
<http://12.0.0.1:5000>
7. Open your regular web browser (like Chrome or Firefox) and navigate to that address. You should now see the " Fetch & Render" interface.

How to Use the Tool

1. URL: Enter the full URL of the webpage you want to fetch (e.g.,
`https://www.example.com`).

2. **User Agent:** Select the User-Agent you want to simulate. You can choose between a desktop or a smartphone Googlebot.
3. **Accept-Language:** Choose the language header to send with the request.
4. **Obey robots.txt:** If checked, the tool will analyze the `robots.txt` file and report any resources that are disallowed for the selected User-Agent.
5. **Render JavaScript:** This is always enabled to ensure the page is fully rendered.
6. **Headless Browser:** If checked, the Chrome browser will run in the background. Unchecking this is useful for debugging, as you will see the browser window open and perform the actions live.
7. **Post-Load Timeout (s):** The number of seconds the tool will wait after the page has loaded before it starts taking screenshots. This is useful for pages that load extra content after a delay.
8. **Click FETCH:** Press the button to start the process. Fetching can take some time depending on the complexity of the target page. Once complete, you will be taken to the results page.

Understanding the Results

The results page is divided into two tabs:

- **HTTP RESPONSE:** This tab shows the HTTP status code (e.g., 200 OK) and all the response headers received from the server for the initial URL request.
- **RENDERED PAGE:** This tab displays the final state of the page after rendering.
 - **Rendered Source:** The full HTML source code of the page *after* all JavaScript has been executed.
 - **Visual Render:** A complete, full-page screenshot. If a smartphone User-Agent was selected, the screenshot will be displayed within a mobile phone frame for better visualization.